

Table 1. Examples of AI safety problems that can be formulated as binary hypothesis testing tasks.

Problem Domain	Alternative Hypothesis ($\mathcal{H}_1$)	Null Hypothesis ($\mathcal{H}_0$)
Jailbreak	Model input attempts to bypass safety guardrails	Legitimate model input
AI-generated Content	AI-generated sample	Real (not AI-generated) sample
Model Fine-tuning	Model updates compromise safety alignment	Model updates are legitimate
Watermark	Data sample is watermarked	Data sample is not watermarked
Membership Inference	A data sample has been used in model training	A data sample has not been used in model training
Data Contamination	A dataset has been used in model training	A dataset has not been used in model training

have their limitations. Rule-based judge functions often fail to generalize in sophisticated situations that require deep contextual understanding or reliability against intentional manipulation, while AI-based solutions may inherit data biases from the training data and exhibit unfairness or hallucination in their judgments (Ye et al., 2025). We refer to the scenario of using an AI-based judge function as a *generative hypothesis testing* problem setting. Verifying and improving the accuracy of the judge functions is an important topic in computational AI safety, but it is also beyond the scope of this paper.

Common Evaluation Metrics Let $\hat{\mathcal{H}}$ denote the predicted hypothesis using a test statistic s and let $\mathcal{H}$ be the “actual” hypothesis determined by a judge function. Let “1” denote True and “0” denote False in binary hypothesis testing. Given an instance, the test result is called a true positive (TP) if $\hat{\mathcal{H}} = \mathcal{H} = 1$, a true negative (TN) if $\hat{\mathcal{H}} = \mathcal{H} = 0$, a false positive (FP) if $\hat{\mathcal{H}} = 1$ but $\mathcal{H} = 0$, and a false negative (FN) if $\hat{\mathcal{H}} = 0$ but $\mathcal{H} = 1$. Extending the evaluation to a set of instances using the same test statistic, and denoting the number of instances of a test result by $\#$. The true positive rate (TPR) is defined as $\text{TPR} = \frac{\#TP}{\#TP + \#FN}$, the false positive rate is $\text{FPR} = \frac{\#FP}{\#FP + \#TN}$, the false negative rate is $\text{FNR} = \frac{\#FN}{\#FN + \#TP}$, and the true negative rate is $\text{TNR} = \frac{\#TN}{\#TN + \#FP}$. Accuracy measures the overall correctness of the predictions, which is defined as the ratio of the correctly tested instances to the total number of instances, $\text{accuracy} = \frac{\#TP + \#TN}{\#TP + \#TN + \#FP + \#FN}$. The equal error rate (EER) is defined as the point on the receiver operating characteristic (ROC) curve where $\text{FPR} = \text{FNR}$, which is equivalent to $\text{EER} = \frac{\text{FPR} + \text{FNR}}{2}$. Furthermore, by varying the threshold η for the test statistic s in hypothesis testing, one can obtain the area under the receiver operating characteristic curve (AUROC), which provides a threshold-independent evaluation of the capability in hypothesis testing and reflects the trade-off between TPR and FPR. Another popular evaluation metric is to report $\text{TPR} @ (\text{FPR} = \alpha\%)$ or $\text{FPR} @ (\text{TPR} = \beta\%)$ for some α and β by setting the threshold to meet the criterion.

3. Hypothesis Testing for Computational AI Safety

With the necessary background and preliminaries introduced in the previous section, we formally present a set of signal processing techniques that can be applied to evaluate and improve computational AI safety. Specifically, Table 1 shows a non-exhaustive list of problems in AI safety that can be formulated as a binary hypothesis testing task in a unified way. Jailbreak is concerned with detecting model inputs that attempt to bypass safety guardrails. AI-generated Content tests whether a data sample is AI-generated. Model Fine-tuning checks if model updates would compromise safety alignment. Watermark verifies if a data sample is watermarked. Membership Inference examines if a data sample has been used for model training, which is highly relevant for machine unlearning. Data Contamination looks into whether a data set has been used for model training, which is critical for ensuring the authenticity of model evaluations on public benchmarks. This paper focuses on the first three problem areas in Table 1. We then present a set of signal processing techniques that can be used to study computational AI safety.

Sensitivity Analysis Inspired by the well-known principle of studying the robustness of signal processing methods in the presence of noise, sensitivity analysis quantifies the amount of change in data representations subject to data manipulation. Let $\mathbf{x}$ be a data sample and let $g(\mathbf{x})$ denote the vectorized continuous latent representation (i.e., embedding) of $\mathbf{x}$ obtained from a deep learning model g . There are two common approaches to measuring sensitivity. The first one is to use a metric $M(g(\mathbf{x}), g(\mathcal{T}(\mathbf{x})))$ to compare the similarity (or difference) between a pair of data embedding $g(\mathbf{x})$ and its manipulated version $g(\mathcal{T}(\mathbf{x}))$, where the function $\mathcal{T}$ denotes a transformation operation on $\mathbf{x}$. In particular, additive Gaussian noise perturbation is a common approach to implement $\mathcal{T}$. The second approach is to compute the gradient of a loss function ℓ evaluated at $g(\mathbf{x})$ (wherever applicable), denoted by $\nabla \ell(g(\mathbf{x}))$, and use the gradient norm $\|\nabla \ell(g(\mathbf{x}))\|$ as a measure of sensitivity. For AI-generated image detection, Ricker et al. implement $\mathcal{T}$ as an image reconstruction function based on an autoencoder and assign M as the reconstruction error for the test statistic (Ricker

et al., 2024). He et al. use an image foundation model for feature extraction and implement additive Gaussian noise for sensitivity analysis to detect AI-generated images (He et al., 2024), where the metric M is the cosine similarity. For jailbreak, Hu et al. compute the gradient norm of an affirmation loss on each token of the model input to identify and mitigate jailbreak prompts (Hu et al., 2025).

Subspace Modeling Subspace modeling involves learning and processing complex (often high-dimensional) data in a compressed low-dimensional subspace. Classic examples include principal component analysis (PCA), sparse signal recovery, subspace tracking, information geometry, and manifold learning, to name a few. In (Hsu et al., 2024), Hsu et al. leverage the technique of subspace projection to mitigate safety degradation during model fine-tuning. Given a pair of base and aligned LLM, denoted as $\{\theta_b, \theta_a\}$, where they share the same neural network architecture, and θ_b is the model obtained after the pre-training phase, whereas θ_a is obtained after fine-tuning θ_b with alignment data. Such a pair of model weights is often publicly available on open-weight model releases. They model the subspace for alignment as the span of the difference between θ_b and θ_a , formally defined as $V = \theta_a - \theta_b$. The weight difference V encapsulates the effect of effort that leads a pre-trained model to become an aligned model. Next, when the aligned model θ_a is to be fine-tuned for a downstream task, incurring a weight change denoted by $\Delta\theta$. Before $\Delta\theta$ is applied to θ_a , the similarity between V and $\Delta\theta$ is checked for each layer in the neural network. If the similarity is above a threshold, indicating that the model update $\Delta\theta$ is aligned with the subspace spanned by V , the model update is performed as is. Otherwise, the model update is performed by first projecting $\Delta\theta$ onto the subspace spanned by V and then adding the projected update to the aligned model θ_a . By doing so, when compared to naive model fine-tuning without subspace projection, possible safety degradation can be mitigated while maintaining high capability on the downstream task.

Loss Landscape Analysis Given a loss function $f : \mathbb{R}^d \mapsto \mathbb{R}$, loss landscape analysis refers to the exploration of the loss function by perturbing the input of f with a set of random and orthogonal directions. For the purpose of loss landscape visualization, the number of perturbations is often limited to 1 or 2 dimensions. The exploration of loss landscape for deep neural networks has facilitated the understanding and advancement of generalization errors, optimization trajectories, and model merging and ensembling (Li et al., 2018; Keskar et al., 2017). Consider the case of a two-dimensional loss landscape visualization centered on a point $\mathbf{u} \in \mathbb{R}^d$ as an input to f . The loss landscape F is obtained by perturbing $\mathbf{u}$ along two random orthogonal unit vectors $\mathbf{v}_1$ and $\mathbf{v}_2$, where α and β represent the respective

coefficients and evaluating them with f , which is defined as

$$F(\alpha, \beta | \mathbf{u}) = f(\mathbf{u} + \alpha \cdot \mathbf{v}_1 + \beta \cdot \mathbf{v}_2). \quad (5)$$

For jailbreak prompt detection, Hu et al. (Hu et al., 2024) define and explore the loss landscape in the space of model input embeddings for eliciting LLM refusals to distinguish benign v.s. malicious user queries. For model fine-tuning, Peng et al. (Peng et al., 2024) use loss landscape analysis to uncover the existence of a “safety basin” in the weight space of an aligned LLM, a bowl-shaped region with a flat loss landscape as measured by jailbreak benchmarks. Such a landscape is unique to safety, as the landscape of capability is drastically different from the shape of a bowl.

Adversarial Learning In the context of safety, adversarial learning refers to the methodology of introducing a virtual adversary to explore and mitigate potential risks of an AI system throughout the development and deployment lifecycle (Chen & Hsieh, 2023), including activities such as creating evaluation benchmarks, identifying vulnerabilities, active testing (dynamic red-teaming), model patching, and system hardening. For example, the virtual adversary can be another AI model, an optimization-based algorithm, or a training environment that provides safety assessment as a reward function. In particular, adversarial robustness measures the worst-case performance of a target AI system in the presence of a virtual adversary, where the interplay between the adversary and the defender (the target AI system to be protected) can be formulated as a two-player game, and the learning of the optimal strategy for each player can be studied through the lens of robust optimization and statistics. To search for effective jailbreak prompts against LLMs, Zou et al. (Zou et al., 2023) formulate this task as an optimization problem that finds the best string of tokens to be inserted as a suffix to user queries. Chao et al. (Chao et al., 2025) use an LLM as the attacker and another LLM as the judge to guide the process of finding successful jailbreak prompts against a target LLM. Xiong et al. (Xiong et al., 2024) propose a jailbreak mitigation strategy by adding a text-based defensive prompt patch after the user query, where the design of such a patch is formulated and solved by casting it as a robust (min-max) optimization problem. For AI-generated text detection, Hu et al. (Hu et al., 2023) train a robust detector by imposing a paraphraser as a virtual adversary.

4. Model Input Safety: Jailbreak Prompt Detection and Mitigation

In general, a jailbreak prompt is a user query (model input) designed to bypass the embedded safety guardrail of a GenAI system, where the system can be as simple as an open-weight LLM, or an integrated closed-source LLM with additional moderation mechanisms in place. Examples of

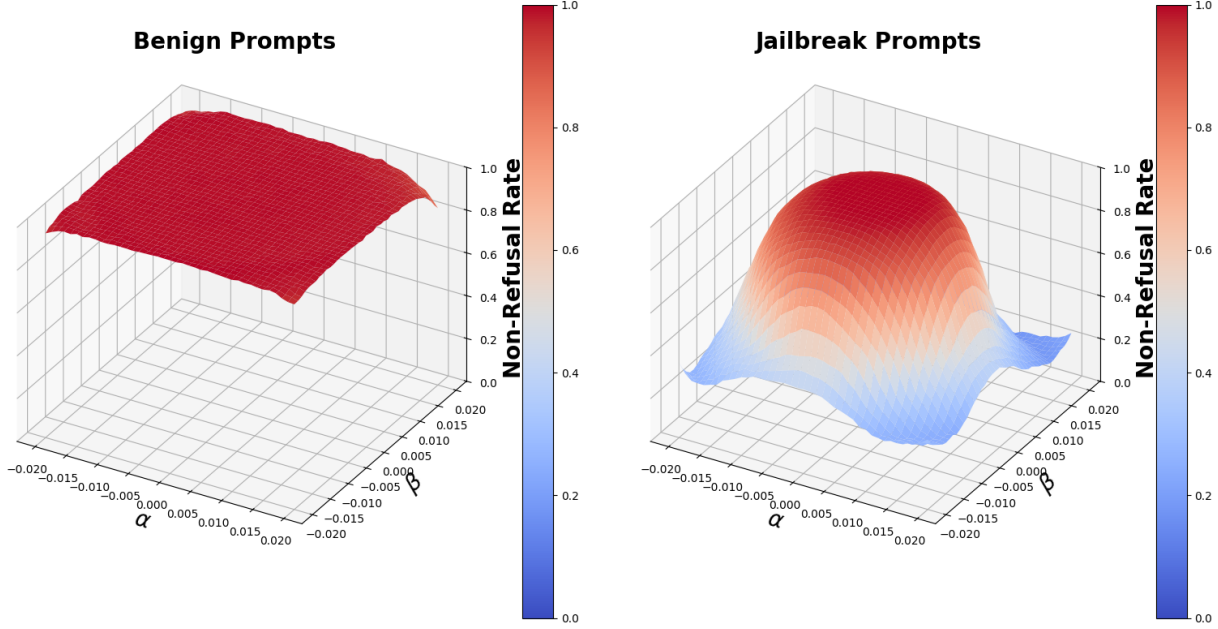


Figure 2. Loss landscape analysis for benign and jailbreak prompts. We use the token embeddings of Vicuna-7B-V1.5 to compute the non-refusal rate of model responses generated from perturbed input embeddings, by interpolating two random directions with additive Gaussian noise in the token embedding space, where the perturbation strengths are denoted by α and β . The results are averaged over 100 prompts. The benign prompts are sampled from Chatbot Arena, and the jailbreak prompts are generated by the greedy coordinate gradient (GCG) attack (Zou et al., 2023). The analysis shows that the jailbreak prompts are more sensitive to Gaussian perturbations than the benign prompts.

jailbreak attack strategies include role-playing, changing system prompts, adding distractive context, and instructing another LLM to design and generate prompts. A judge function inspects the output of the target GenAI system to evaluate the success of a jailbreak prompt. A rule-based judge function uses keyword matching and declares the attack in vain if the output contains some keywords like “Sorry”, “I can’t fulfill”, etc. However, such a judgment lacks a comprehensive understanding of the context and might overlook jailbreaks that force the target model to respond in an apologetic tone. On the other hand, one can use another LLM or a customized LLM (i.e., an LLM guard model) as a judge to evaluate whether the output is safe or unsafe. While LLM-based judge functions possess contextual awareness, they still lack transparency about the decision and may have their own biases. Nevertheless, despite their limitations, these judge functions are poised as hypothesis-testing verifiers in our computational AI safety framework.

Safety-Capability Trade-off When defending against jailbreak attempts, an implicit requirement of a practical mitigation approach is the ability to effectively identify and thwart jailbreak prompts while minimizing the impact on the capability of the protected model. A model that is over-aligned to cause over-rejection of benign requests (i.e., high safety but low capability), or performant but under-aligned

to be easily jailbroken (i.e., high capability but low safety), is undesirable. Therefore, exploring and optimizing the safety-capability trade-off of a deployed jailbreak mitigation is essential to computational AI safety. Next, we present two methods for detecting and mitigating jailbreak prompts to achieve a good safety-capability tradeoff.

Detecting Jailbreak Prompts via Loss Landscape Analysis

In (Hu et al., 2024), Hu et al. apply loss landscape analysis to the embedding of jailbreak prompts and the responses of an LLM. They show that jailbreak prompts exhibit different characteristics than benign (safe) prompts in the landscape studied, and this finding can be used to detect jailbreak prompts. Specifically, given a prompt $\mathbf{x}$ of token length $|\mathbf{x}|$, an LLM first embeds the tokens of $\mathbf{x}$ into a token embedding $\mathbf{E}(\mathbf{x}) \in \mathbb{R}^{|\mathbf{x}| \times d}$, where d is the embedding dimension. The function g denotes an LLM, and $g(\mathbf{E}(\mathbf{x}))$ means the response of g given the context $\mathbf{x}$. A judge function $J(g(\mathbf{E}(\mathbf{x})), \mathbf{x}) \in \{0, 1\}$ is used for safety verification, where “1” means $\mathbf{x}$ is a successful jailbreak prompt that causes harmful behavior of g , and “0” otherwise. Finally, Hu et al. explore the loss landscape around $J(g(\mathbf{E}(\mathbf{x})), \mathbf{x})$ by defining a non-refusal loss on $\mathbf{x}$ as $f(\mathbf{x}|g, J) = 1 - \frac{1}{N} \sum_{i=1}^N J(g^{(i)}(\mathbf{E}(\mathbf{x})), \mathbf{x})$, which signifies the average non-refusal rate over N independent inferences of g on $\mathbf{x}$ (each inference is indexed